Stats 21 - HW 2

[Ayla Lee]

The questions have been entered into this document. You will modify the document by entering your code.

Make sure you run the cell so the requested output is visible. Download/export it as an HTML file and then print to PDF. Be sure to tag your pages on Gradescope.

Homework is an opportunity to practice coding and to practice problem solving. Doing exercises is where you will do most of your learning.

Using AI or copying someone else's solutions takes away your learning opportunities. It is also academic dishonesty.

Reading

- Think Python: Chapters 6 through 10

Reading is important! Keep up with the reading. I recommend alternating between reading a chapter and then working on exercises.

Additional recommended reading:

- String methods documentation https://docs.python.org/3/library/stdtypes.html#string-methods

Textbook Chapter 5 Problems

Modified Exercise 5.14.2

https://allendowney.github.io/ThinkPython/chap05.html#exercise

In [ ]:
```
import time
```

In [ ]:
```
time.time()
```

Out[ ]:
```
1776916404.503212
```

Write a function `now()` that reads the current time and prints out the time of day in hours, minutes, and seconds, plus the number of days since the epoch. The function does not need to return a value, just print output to the screen.

The result should look like:

"Current time is: 15:25:47. It has been 18370 days since the epoch."

Use `int()` to drop decimal values. You do not need to try to find the date with years and months.

Tip: build your function incrementally. Start by finding how many days have passed since the epoch. (check your answer at the bottom of the page: https://www.epochconverter.com/seconds-days-since-y0) From there find how many hours, etc. Keep in mind the hours will be UTC time.

In [ ]:
```
import time
def now():
    # Get the current time in total seconds since the epoch
    total_seconds = time.time()

    # 1 day = 24 hours * 60 minutes * 60 seconds = 86400 seconds
    # Calculate the total number of days passed using floor division
    days = int(total_seconds // 86400)

    # Calculate the remaining seconds for the current day using modulo
    remaining_seconds = total_seconds % 86400

    # 1 hour = 60 minutes * 60 seconds = 3600 seconds

    hours = int(remaining_seconds // 3600)

    # Update remaining seconds after extracting hours
    remaining_seconds = remaining_seconds % 3600

    # 1 minute = 60 seconds
    # Calculate current minutes
    minutes = int(remaining_seconds // 60)

    # The final remaining seconds
    seconds = int(remaining_seconds % 60)

    # The :02d format ensures numbers like '5' are printed as '05' for prope
    print(f"Current time is: {hours:02d}:{minutes:02d}:{seconds:02d}. It has
```

In [ ]:
```
now()
```

```
Current time is: 03:53:24. It has been 20566 days since the epoch.
```

Textbook Chapter 6 Problems

Modified Exercise 6.12.4

https://allendowney.github.io/ThinkPython/chap06.html#id2

The Ackermann function, A(m, n), is defined:

$$A(m,n) = \begin{cases} n+1 & \text{if } m = 0 \\ A(m-1,1) & \text{if } m > 0 \text{ and } n = 0 \\ A(m-1, A(m, n-1)) & \text{if } m > 0 \text{ and } n > 0 \end{cases}$$

See http://en.wikipedia.org/wiki/Ackermann_function . Write a function named `ack` that evaluates the Ackermann function. Use your function to evaluate a few test cases. Don't test with $m \geq 4$ as it grows very fast very quickly.

In [ ]:
```
def ack(m,n):
# Condition 1: if m is 0
    if m == 0:
        return n + 1

    # Condition 2: if m > 0 and n is 0
    # We already know m > 0 if the code reaches here, so we only check n ==
    if n == 0:
        return ack(m - 1, 1)

    # Condition 3: if m > 0 and n > 0
    # If the code reaches here, it means m > 0 and n > 0
    return ack(m - 1, ack(m, n - 1))
```

In [ ]:
```
# test case, should be 61
ack(3, 3)
```

Out[ ]: 61

In [ ]:
```
# test case, should be 125
ack(3, 4)
```

Out[ ]: 125

Modified Exercise based on Think Python 2ed. Chap 6, Section 11, Exercise 6.4

A number, `a`, is a power of `b` if it is divisible by `b` and `a/b` is a power of `b`. Write a function called `is_power` that takes parameters `a` and `b` and returns `True` if a is a power of b. Note: you will have to think about the base case.

In [ ]:
```
def is_power(a, b):
    if a == b:
        return True
```

```
    # Edge case: To prevent infinite recursion when b is 1
    # Also, if a is not divisible by b, it can't be a power
    if b == 1 or a % b != 0:
        return False

    return is_power(a // b, b)
```

In [ ]:
```
is_power(1024, 2)
```

Out[ ]: True

In [ ]:
```
is_power(6561, 3)
```

Out[ ]: True

In [ ]:
```
is_power(4374, 3)
```

Out[ ]: False

In [ ]:
```
is_power(768, 2)
```

Out[ ]: False

Modified Exercise 6.12.5

https://allendowney.github.io/ThinkPython/chap06.html#id3

The greatest common divisor (GCD) of a and b is the largest number that divides both of them with no remainder.

One way to find the GCD of two numbers is based on the observation that if `r` is the remainder when `a` is divided by `b`, then `gcd(a, b) = gcd(b, r)`.

As a base case, we can use `gcd(a, 0) = a`.

Write a function called `gcd` that takes parameters `a` and `b` and returns their greatest common divisor.

In [ ]:
```
def gcd(a, b):
    # Base case: If b is 0, the GCD is a
    if b == 0:
        return a

    # Recursive case: a % b gives the remainder (r)
    return gcd(b, a % b)
```

In [ ]:
```
gcd(21, 7)
```

Out[ ]: 7

In [ ]:
```
gcd(42, 28)
```

Out[ ]: 14

In [ ]:
```
gcd(105, 140)
```

Out[ ]: 35

Textbook Chapter 7 Problems

Exercise 7.9.2

https://allendowney.github.io/ThinkPython/chap07.html#exercise

Write a function named `uses_none` that takes a word and a string of forbidden letters, and returns `True` if the word does not use any of the forbidden letters.

Here's an outline of the function that includes two doctests. Fill in the function so it passes these tests, and add at least one more doctest.

In [ ]:
```
def uses_none(word, forbidden):
    """Checks whether a word avoid forbidden letters.

    >>> uses_none('banana', 'xyz')
    True
    >>> uses_none('apple', 'efg')
    False
    """
    for letter in word:
        # If the current letter is inside the 'forbidden' string, return Fal
        if letter in forbidden:
            return False

    return True
```

In [ ]:
```
uses_none('banana', 'xyz')
```

Out[ ]: True

In [ ]:
```
uses_none('apple', 'efg')
```

Out[ ]: False

Exercise 7.9.3.

https://allendowney.github.io/ThinkPython/chap07.html#id1

Write a function called `uses_all` that takes a word and a string of letters, and that returns `True` if the word contains all of the letters in the string at least once.

Here's an outline of the function that includes two doctests. Fill in the function so it passes these tests, and add at least one more doctest.

In [ ]:
```
def uses_all(word, required):
    """Checks whether a word uses all required letters.

    >>> uses_all('banana', 'ban')
    True
    >>> uses_all('apple', 'api')
    False
    """
    for letter in required:
        # If a required letter is NOT in the word, return False immediately
        if letter not in word:
            return False

    return True
```

In [ ]:
```
uses_all('banana', 'ban')
```

Out[ ]: True

In [ ]:
```
uses_all('apple', 'api')
```

Out[ ]: False

Problems from Think Python 2nd Ed. Chapter 9

Exercise 9.1

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 84.

Write and run a script that reads `words.txt` and prints out only the words with more than 20 characters (after stripping whitespace).

In [ ]:
```
fin = open("words.txt")
for line in fin:
    pass
```

Exercise 9.2

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 84.

Write a function called `has_no_e` that returns True if the word doesn't have the letter e. You can use any of Pythons availble string methods.

In [ ]:
```
def has_no_e(text):
    return 'e' not in text
```

In [ ]:
```
has_no_e("hello")
```

Out[ ]:
```
False
```

In [ ]:
```
has_no_e("quit")
```

Out[ ]:
```
True
```

With your function, write a script. The script should read the list of words (`words.txt`), print out the number of words that do not have the letter 'e' and the proportion of words that do not have the letter 'e'

In [ ]:
```
fin = open("words.txt")
total_words = 0
no_e_words = 0

for line in fin:
    word = line.strip()
    total_words += 1

    if has_no_e(word):
        no_e_words += 1

if total_words > 0:
    proportion = (no_e_words / total_words) * 100
else:
    proportion = 0
```

Problems from Think Python 2nd Ed. Chapter 10

Exercise 10.1

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 100

Write a function called `nested_sum` that takes a list of lists of integers and adds up the elements from all of the nested lists. For example:

```
t = [[1, 2], [3], [4, 5, 6]]
nested_sum(t)
```

```
21
```

You may want to build the function recursively in case there are many levels of nested lists.

You can assume that all elements in any of the nested lists are numeric.

In [ ]:
```
def nested_sum(t):
    total = 0

    for item in t:
        # If the item is a list, recursively call nested_sum to get its sum
        if isinstance(item, list):
            total += nested_sum(item)
        # If the item is a number (integer/float), just add it to the total
        else:
            total += item

    return total
```

In [ ]:
```
t = [1, 2]
nested_sum(t)
```

Out[ ]: 3

In [ ]:
```
t = [[1, 2], [3], [4, 5, 6]]
nested_sum(t)
```

Out[ ]: 21

In [ ]:
```
x = [[1, 2, [3]], 4, 5, 6, [7], 8]
nested_sum(x)
```

Out[ ]: 36

In [ ]:
```
t = [[[1, 2, [3]], [4, [5, 6, [7]], 8]]]
nested_sum(t)
```

Out[ ]: 36

Exercise 10.2

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 100

Write a function called `cumsum` that takes a list of numbers and returns the cumulative sum; that is, a new list where the ith element is the sum of the first i + 1 elements from the original list.

For example:

```
t = [1, 2, 3]
cumsum(t)
[1, 3, 6]
```

You can assume that all elements in the lists are numeric and the list does not contain nested lists.

In [ ]:
```
def cumsum(t):
# This will hold our final list of cumulative sums
    result = []
    # This variable keeps track of the running total
    total = 0

    # Iterate through each number in the input list/iterable
    for num in t:
        # Add the current number to the running total
        total += num
        result.append(total)

    return result
```

In [ ]:
```
cumsum([1, 2, 3, 4])
```

Out[ ]:
```
[1, 3, 6, 10]
```

In [ ]:
```
cumsum(range(12))
```

Out[ ]:
```
[0, 1, 3, 6, 10, 15, 21, 28, 36, 45, 55, 66]
```

Exercise 10.6

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 101

Two words are anagrams if you can rearrange the letters from one to spell the other. Write a function called `is_anagram` that takes two strings and returns `True` if they are anagrams.

You can remove spaces and convert to lowercase using `string.replace(" ","").lower()`

In [ ]:
```
def is_anagram(word1, word2):
# Remove spaces and convert to lowercase for both words
    clean_word1 = word1.replace(" ", "").lower()
    clean_word2 = word2.replace(" ", "").lower()

    # If they have the exact same letters, sorting them will result in the s
    return sorted(clean_word1) == sorted(clean_word2)
```

In [ ]:
```
is_anagram("hello", "o hell")
```

Out[ ]: True

In [ ]:
```
is_anagram("dormitory" , "dirty room")
```

Out[ ]: True

In [ ]:
```
is_anagram("dormitory" , "dirty rooms")
```

Out[ ]: False

In [ ]:
```
is_anagram("astronomers" , "moon starers")
```

Out[ ]: True

Exercise 10.7

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 101

Write a function called `has_duplicates` that takes a list and returns `True` if there is any element that appears more than once. It should not modify the original list.

You can assume that the list will not have nested lists.

In [ ]:
```
def has_duplicates(t):
    return len(t) != len(set(t))
```

In [ ]:
```
has_duplicates(['a','b','c'])
```

Out[ ]: False

In [ ]:
```
has_duplicates(['a','b','b','c'])
```

Out[ ]: True

In [ ]:
```
has_duplicates(['a','b','c','a'])
```

Out[ ]: True

Exercise 10.10

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 101

To check whether a word is in the word list, you could use the in operator, but it would be slow because it searches through the words in order.

Because the words are in alphabetical order, we can speed things up with a bisection search (also known as binary search). You start in the middle and check to see whether the word you are looking for comes before the word in the middle of the list. If so, you search the first half of the list the same way (perform a bisection search on the first half). Otherwise you search the second half.

Either way, you cut the remaining search space in half. If the word list has 113,809 words, it will take about 17 steps to find the word or conclude that it's not there.

Write a function called `in_bisect` that takes a sorted list and a target word and will returns `True` if the word is in the list and `False` if it's not.

Hint: it's a recursive function.

In [ ]:
```
# Use this function. No need to rewrite it.
def make_word_list():
    """Reads lines from a file and builds a list."""
    t = []
    fin = open('words.txt')
    for line in fin:
        word = line.strip()
        t.append(word)
    return t

t = make_word_list()
```

In [ ]:
```
# define this function
def in_bisect(word_list, word):
# Base case 1: If the list is empty, the word is not in the list
    if len(word_list) == 0:
        return False

    # Find the middle index of the current list
    mid = len(word_list) // 2

    # Base case 2: If the middle word is the target word, we found it!
    if word_list[mid] == word:
        return True

    # Recursive case 1: If the target word comes BEFORE the middle word alph
    # Search only the left half of the list
    if word < word_list[mid]:
        return in_bisect(word_list[:mid], word)

    # Recursive case 2: If the target word comes AFTER the middle word alpha
    # Search only the right half of the list
    else:
        return in_bisect(word_list[mid+1:], word)
```

In [ ]:
```
in_bisect(t, "hello")
```

Out[ ]: True

In [ ]:
```
in_bisect(t, "xyz")
```

Out[ ]: False

Exercise 10.11

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 101

Two words are a "reverse pair" if each is the reverse of the other.

Now that you have the `in_bisect` search, write a script that finds all the reverse pairs in the word list that are 6 letters or longer. (It takes a little bit of time to run.)

In [ ]:
```
### ('agenes', 'senega')
### ('animal', 'lamina')
### ...
### ('lamina', 'animal')
### ...
### ('senega', 'agenes')
### Because of the nature of a reverse pair, each pair will appear twice in
```

In [ ]:
```
def find_reverse_pairs(word_list):
    # Iterate through each word in the list
    for word in word_list:

        # Condition 1: The word must be 6 letters or longer
        if len(word) >= 6:

            # Reverse the word using Python string slicing
            reversed_word = word[::-1]

            # Condition 2: Check if the reversed word is also in the list
            if in_bisect(word_list, reversed_word):
                print((word, reversed_word))

t = make_word_list()
find_reverse_pairs(t)
```

```
('agenes', 'senega')
('animal', 'lamina')
('animes', 'semina')
('degami', 'imaged')
('deified', 'deified')
('deifier', 'reified')
('deliver', 'reviled')
('denier', 'reined')
('denies', 'seined')
('denned', 'denned')
('depots', 'stoped')
('derats', 'stared')
('dessert', 'tressed')
('desserts', 'stressed')
('dewans', 'snawed')
('dialer', 'relaid')
('diaper', 'repaid')
('dormin', 'nimrod')
('drawer', 'reward')
('elides', 'sedile')
('eviler', 'relive')
('gelder', 'redleg')
('halalah', 'halalah')
('hallah', 'hallah')
('imaged', 'degami')
('lamina', 'animal')
('levins', 'snivel')
('looter', 'retool')
('marram', 'marram')
('nimrod', 'dormin')
('pupils', 'slipup')
('recaps', 'spacer')
('redder', 'redder')
('redips', 'spider')
('redleg', 'gelder')
('redraw', 'warder')
('redrawer', 'rewarder')
('reflet', 'telfer')
('reflow', 'wolfer')
('reified', 'deifier')
('reifier', 'reifier')
('reined', 'denier')
('reknit', 'tinker')
('reknits', 'stinker')
('relaid', 'dialer')
('relive', 'eviler')
('remeet', 'teemer')
('rennet', 'tenner')
('repaid', 'diaper')
('repaper', 'repaper')
('repins', 'sniper')
('retool', 'looter')
('reviled', 'deliver')
('reviver', 'reviver')
('reward', 'drawer')
('rewarder', 'redrawer')
```

```
('rotator', 'rotator')
('sallets', 'stellas')
('scares', 'seracs')
('secret', 'terces')
('sedile', 'elides')
('seined', 'denies')
('selahs', 'shales')
('selles', 'selles')
('sememes', 'sememes')
('semina', 'animes')
('senega', 'agenes')
('seracs', 'scares')
('shales', 'selahs')
('skeets', 'steeks')
('sleeps', 'speels')
('sleets', 'steels')
('slipup', 'pupils')
('sloops', 'spools')
('snawed', 'dewans')
('sniper', 'repins')
('snivel', 'levins')
('snoops', 'spoons')
('spacer', 'recaps')
('speels', 'sleeps')
('spider', 'redips')
('spirts', 'strips')
('spools', 'sloops')
('spoons', 'snoops')
('sports', 'strops')
('sprits', 'stirps')
('stared', 'derats')
('steeks', 'skeets')
('steels', 'sleets')
('stellas', 'sallets')
('stinker', 'reknits')
('stirps', 'sprits')
('stoped', 'depots')
('stressed', 'desserts')
('strips', 'spirts')
('strops', 'sports')
('struts', 'sturts')
('sturts', 'struts')
('teemer', 'remeet')
('telfer', 'reflet')
('tenner', 'rennet')
('terces', 'secret')
('terret', 'terret')
('tinker', 'reknit')
('tressed', 'dessert')
('warder', 'redraw')
('wolfer', 'reflow')
```